

★ PROJECT PROBLEM STATEMENT

Task 3 — Real-Time Character Counter App

📌 1. Project Title

★ "Real-Time Character Counter & Text Analyzer App"

📌 2. Problem Statement

You are required to build a **React-based Real-Time Text Analyzer** that provides instant feedback as the user types. The application should track the **character count**, **word count**, and **limit warnings**, and should have intelligent features such as **auto-focus**, **live updates**, **optimizations**, and **warning messages**.

The project must make use of:

- ✓ useState
- ✓ useEffect
- ✓ useRef
- ✓ useCallback
- ✓ Props

This app will simulate how modern writing tools (like Twitter, Instagram Bio, LinkedIn posts) enforce limits and provide real-time feedback.

📌 3. Objective

Students will learn how to build a responsive interactive component using multiple React Hooks, understand their use-cases, and create a connected component structure using props.

📌 4. Learning Outcomes

After completing this project, the student will:

- ✓ Understand how to manage text input state
- ✓ Use useRef to access DOM elements
- ✓ Use useEffect to trigger warnings and updates
- ✓ Use useCallback to optimize functions
- ✓ Use props to pass data between components

- ✓ Understand re-rendering behavior of hooks
 - ✓ Build a clean UI React component system
-

5. Core Requirements (Mandatory)

Your app must include the following:

A. Input Box Component

- Auto-focus on load → using **useRef**
- Tracks the user's input → using **useState**
- Fires analysis every time text changes → **useEffect**
- Button to clear input → uses **useRef** to reset field

B. Real-Time Character Counter

- Shows Total Characters
- Shows Remaining Characters (Out of 200 limit)
- Turns **yellow** when 80% limit reached
- Turns **red** when crossing the limit
(Use conditional CSS based on state)

C. Real-Time Word Counter

- Count words using simple split logic
- Show updates instantly

D. Warning Component (Via Props)

- A separate `<Warning />` component
- Receives message via **props**
- Displays warning using animation when needed

E. useCallback Optimization

- Memoize the `analyzeText` function using `useCallback`
 - Ensure function is not recreated on each re-render
-

6. Additional Functionalities (Mandatory)

✓ Live Preview Section

Shows the entered text in real-time (props).

✓ Analysis Summary Component

Displays:

- Total Characters
- Total Words
- Special Characters Count
- Limit Status

All via props.

🔪 7. Advanced Tasks (Optional but Recommended)

Students can earn extra marks by implementing:

★ Debounce (delay) user typing

- Text analysis happens after 300ms of user stopping typing
- Uses **useEffect + setTimeout + useRef**

★ Dark Mode Toggle

- Store mode in state
- CSS theme change

★ Custom Hook

useAnalyzeText()

- Returns all counts and analysis
- Used inside main component

★ Save text in LocalStorage

- Restores last typed text on reload
 - Using **useEffect**
-

🔪 8. Component Structure (Mandatory)

Your project MUST include at least these components:

```
App.jsx
├─ TextInput.jsx
├─ AnalysisPanel.jsx
├─ Warning.jsx
└─ Preview.jsx
```

9. Sample Flow of Application

1. App loads → Input auto-focuses
2. User types → useState updates
3. useEffect monitors text changes
4. useCallback memoizes analysis function
5. Warning component shows alert if limit reached
6. AnalysisPanel updates instantly
7. Preview component shows live text

10. Submission Checklist

- ✓ Project runs without errors
- ✓ At least 5 hooks used
- ✓ Console logs added to understand hook behavior
- ✓ Clean CSS & responsive UI
- ✓ Code commented properly
- ✓ Components divided logically
- ✓ Props used correctly